Informatique
Faculté des science
Université de Sherbrooke

Travail Pratique 1 - Rapport

Par Nicolas Claveau-Chaput

Destiné au professeur Maxime Descoteaux
dans le cadre du cours IMN 530 - Reconstruction et analyse d'images médicales

Sherbrooke
11 octobre 2026

Introduction

Ce rapport couvre les questions requises avec la remise du travail pratique (portion rapport), ainsi que deux sections supplémentaires. Une sur mon raisonnement au cours de la programmation du projet (Notes sur la portion programmation) et une autre sur une explication de mon utilisation de l'IAG (Copilot) pour ce travail (Utilisation de l'IAG).

Autre chose à savoir : j'ai commencé à travailler sur ce travail pratique la fin de semaine du 12 septembre. Mon raisonnement et ma compréhension ont donc grandement évolué au fur et à mesure du travail, considérant que ce travail m'a grandement « remis dans le bain » de la programmation en Python.

Portion rapport
Question 1

Numéro 1 b)

Créez un script compute_mIP_MIP.py qui calcule le maximum et minimum intensity projection le long d'un axe. L'axe choisi, et l'option (min ou max) doivent être des arguments du script.
Indice: Utilisez votre viewer pour déterminer les paramètres intéressants! Justifiez dans votre rapport.

En explorant un peu le code de mon viewer, la documentation en ligne et en questionnant Copilot, les paramètres intéressants pour l'implémentation du compte étaient ceux en lien avec les fonctions np.max, np.min et plt.imshow.

Pour np.max et np.min, ces fonctions sont capables de prendre comme paramètre un axe avec l'argument axis. Le paramètre doit être un nombre, soit 0, 1 ou 2, représentant respectivement les axes x, y ou z. Cet argument précise sur quel axe NumPy doit aller chercher ses informations.

Dans ce code en particulier, les fonctions np.max et np.min parcourent toutes les valeurs le long de l'axe choisi et retournent une image. Pour chaque pixel de cette image, np.max garde l'intensité maximale, tandis que np.min conserve l'intensité minimale. Tout cela permet de générer respectivement soit une Maximum Intensity Projection (MIP), soit une Minimum Intensity Projection (MinIP).

Ces projections d'intensité sont vraiment utiles pour être capable de mieux observer les structures présentes dans les images, ce qui sera particulièrement utile pour la question 2c de ce travail.

Pour plt.imshow, puisque la fonction de base de cet outil est d'afficher une image à partir d'une entrée de données correspondant à une image standard, j'ai simplement pu passer en paramètre l'image de projection, correspondant soit au maximum, soit au minimum de l'axe désiré selon l'appel effectué avec l'argparser.

Le paramètre cmap="gray" est aussi pertinent à ajouter dans cette fonction puisqu'il peut aider à la visibilité des différentes structures. Je l'ai tout de même rendu facultatif afin de laisser le choix du mode d'affichage des couleurs selon les préférences de l'utilisateur.

Numéro 1 c)

Indiquez dans votre rapport l'orientation de vos images (LPI / RAS / etc) sur disque (ex. mrinfo).
Vérifiez l'orientation de l'image une fois « loadée » avec nibabel ?

Le code pour trouver les orientations est situé dans view_image.py. Tel qu'attendu, les résultats entre MrInfo et Nibabel concordent pour la grande majorité des fichiers NIfTI fournis.

Cependant, les fichiers Stack.nii et Slice06.nii présentent des résultats différents entre MrInfo et Nibabel. Après discussion avec Charles, cette différence serait probablement due à une corruption ou à une erreur dans l'en-tête des fichiers. Il est difficile d'établir si l'un des lecteurs est plus fiable que l'autre puisqu'ils interprètent l'en-tête de façon différente.

IRM\Brain\t1.nii MrInfo orientation: R, A, S Nibabel orientation: R, A, S	**Ultrasound\us.nii** MrInfo orientation: R, A, S Nibabel orientation: R, A, S
IRM\Brain\flair.nii MrInfo orientation: R, A, S Nibabel orientation: R, A, S	**IRM\Bone\Stack.nii** MrInfo orientation: R, A, S Nibabel orientation: L, A, S
IRM\Brain\fa.nii MrInfo orientation: R, A, S Nibabel orientation: R, A, S	**IRM\Heart\PetitAxe\Slice06.nii** MrInfo orientation: R, A, S Nibabel orientation: L, A, S

Question 2

Numéro 2 b) et c)

Pour le fichier rat111.vff, j'ai utilisé un plugin pour Fiji afin de convertir le fichier en NIfTI afin qu'il soit utilisable dans les outils d'analyse de show_image_stats.py. (https://imagej.net/ij/plugins/nifti.html)

Avant de commencer les questions, j'aimerais ajouter que l'échographie que nous avons dû analyser est de très faible qualité, autant sur le plan visuel qu'au niveau des données. Et, je cite ma mère : « Je n'engagerais pas la personne qui a fait cette échographie-là. »

i) Taille des voxels et taille de chaques images

La taille des voxels ainsi que la taille des images est indiqué dans les entêtes des fichiers.

	Taille des voxels	**Taille des images**
Epaule.pgm	La taille des voxels est inexistante sur ce fichier puisque les fichiers de format .pgm sont des images 2D natives. Quoi que certains fichier .pgm peuvent inclure une taille de voxel dans l'entête, ce n'est pas le cas pour celui-ci. Par technicalité, on peut quand même dire que la taille des voxels est 1x1x1x0.	512x 512 x 1 voxels
rat111.vff	0.100401 x 0.100401 x 0.100401 mm (La conversion vers nifti semble avoir fait des drôles de choses à la taille des voxels, ils sont affichés comme étant tous 100.401 mm quand ils sont en réalité 0.100401 mm)	384 x 433 x 532 x 1 voxels
Flair.nii	0.820 x 0.820 x 7.425 mm	208 x 256 x 22 x 1 voxels
Heart (Slice06.nii)	0.195 x 0.195 x 1.000 mm	256 x 256 x 14 x 1 voxels
T1.nii	1.0 x 1.0 x 1.0 mm	192 x 256 x 176 x 1 voxels
us.nii	0.300 x 0.300 x 0.300 mm	323 x 366 x 371 x 1 voxels

ii) Contraste de Michelson et RMS

	Contraste de Michelson	**RMS**
Epaule.pgm	1.000000	34.344110
rat111.vff	3.178171	587.864899
Flair.nii	1.000000	191.872844
Heart (Slice06.nii)	0.999980	0.023725
T1.nii	1.000000	161.311491
us.nii	1.000000	0.000373

Le contraste de Michelson est proche ou égal à 1 pour la majeure partie de nos images. Cela indique que chacune d'entre elles possède une intensité minimale nulle. Par conséquent, le contraste de Michelson ne nous permet pas de distinguer efficacement les images de l'ensemble puisqu'il produit la même valeur dans la majorité des cas.

Pour ce qui est du Root Mean Square (RMS), plus le RMS est élevé, plus les niveaux de gris sont dispersés et plus l'image est contrastée. Par conséquent, on peut observer que us.nii et Heart ont des valeurs très faibles, ce qui signifie qu'il y a peu de variation d'intensité dans ces images.

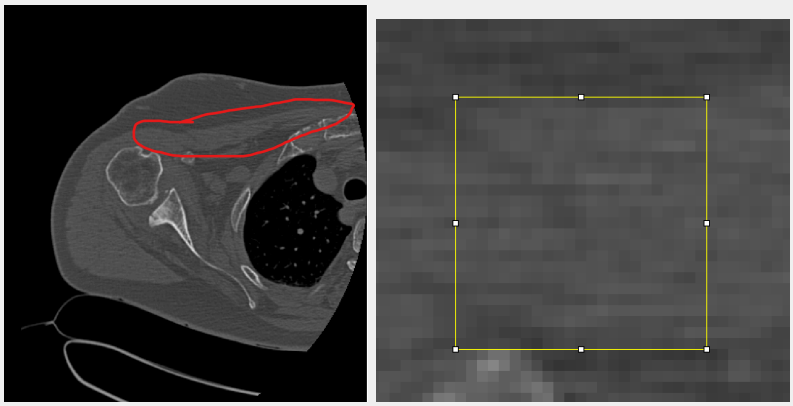
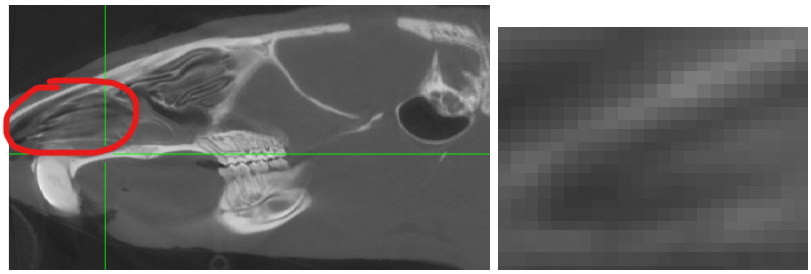
Les autres résultats pour le RMS donnent des valeurs plus élevées, ce qui indique qu'il y a davantage de variation entre les quantités de hautes et de basses intensités.

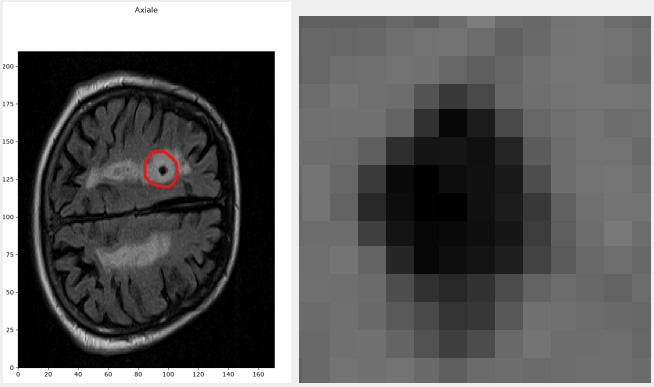
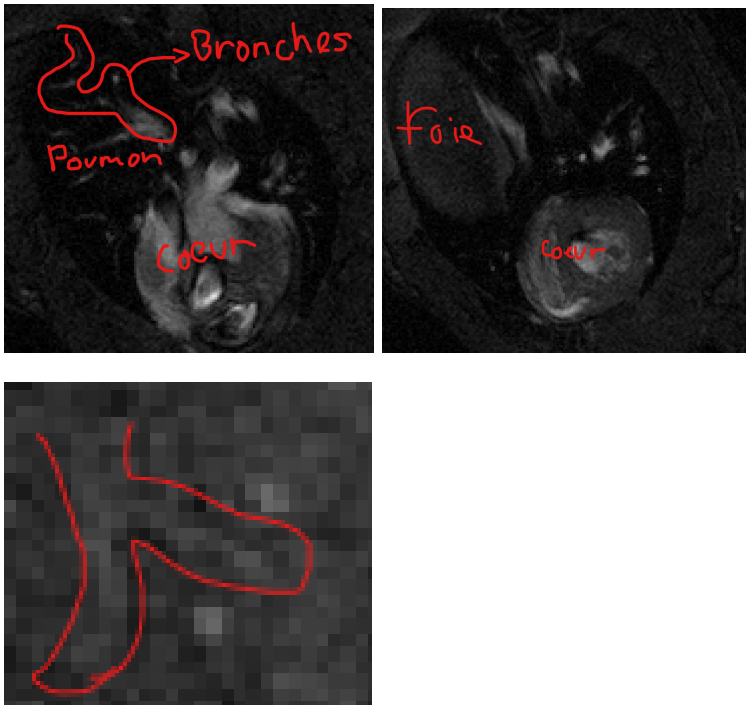
iii) Taille de la plus petite structure détectable dans l'image.

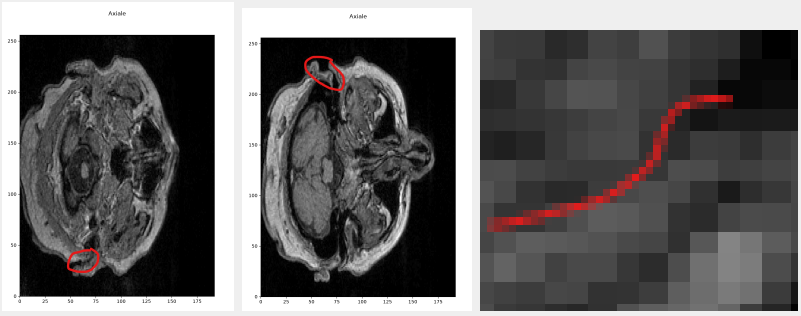
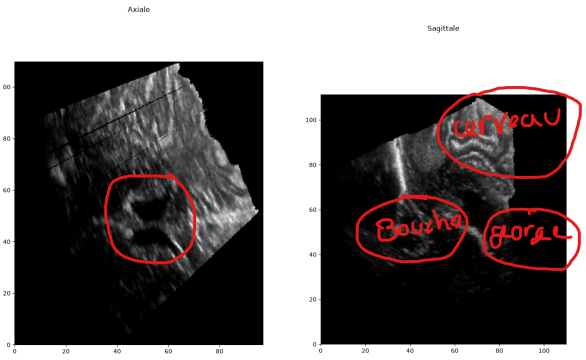
Il est possible de déterminer la taille des structures en comptant le nombre de voxels ou de pixels et en les multipliant par leur taille réelle, telle qu'indiquée dans les en-têtes des différents fichiers.

Cette partie de l'analyse a été réalisée en collaboration avec ma mère. Mes connaissances en anatomie remontent au cégep.

Son aide a donc été grandement utile pour comprendre ce que représentaient les images.

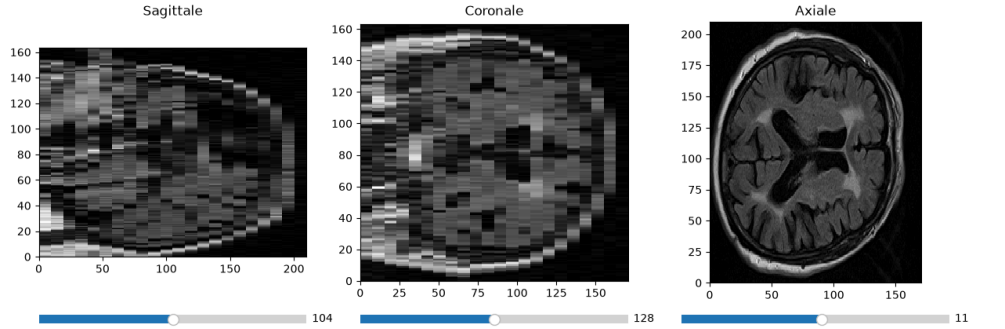
	Taille de la plus petite structure détectable dans l'image.
Epaule.pgm	La personne semble être couchée sur le dos. On est capable de distinguer la clavicule, mais un expert serait certainement capable d'observer une plus petite structure. Sa taille est d'une dizaine de pixel en moyenne ce qui donne un diamètre d'environ 10 mm et plus. 
rat111.vff	La plus petite structure observable serait possiblement les canaux nasals. Leur taille serait d'environ de 3 pixel de diamètre soit environ 0.45 mm. 

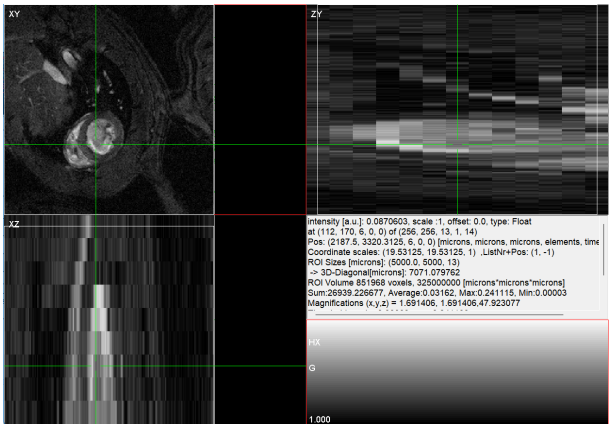
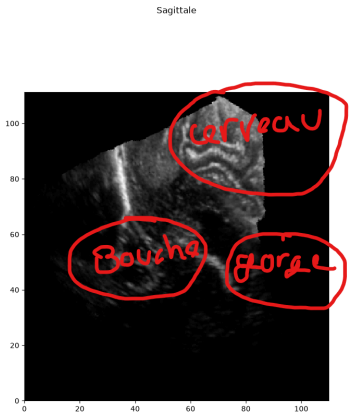
Flair.nii	La kyste semble être la plus petite structure visible. Cependant, l'épaisseur sur la coupe axiale et le manque de clarté sur les autres coupes rend cette observation difficile à confirmer. Selon le metadata de l'image, la taille du kyste serait de 4 pixel de diamètre en moyenne soit un peu moins de 3.5 mm. 
Heart (Slice06.nii)	En se basant sur les structures environnantes, on est capable de distinguer différents organes comme le foie, le coeur et les poumons. Dans les poumons, on perçoit les bronches et les bronchioles. Leurs tailles allant de 0.1 mm à 0.3 mm. 

T1.nii	Les canaux internes des oreilles semblent être les plus petites structures facilement distinguables dans les images. Selon le metadata des images, leur taille serait de 4-5 pixel soit 4-5 mm. 
us.nii	L'ultrason est très difficile à analyser. De ce qu'on peut observer, l'ultrason semble être celle du visage/cerveau d'une personne. On est capable de distinguer les ventricules du cerveau. La taille des ventricules serait d'une 20aine de pixel soit environ 10 mm. 

iv) Effets de volume partiel

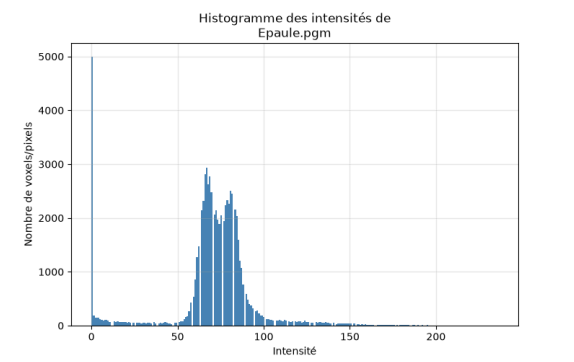
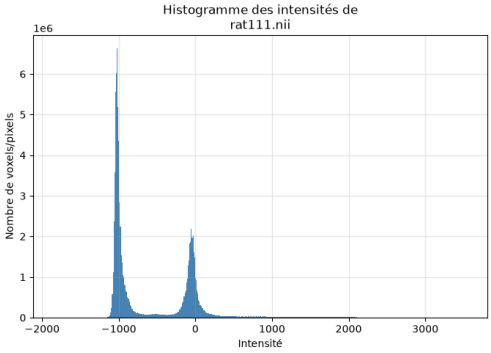
Tel que vu dans les notes de cours, les effets de volume partiel apparaissent lorsque les structures présentes dans les images sont plus petites que la taille des voxels. Cela rend les images difficiles à analyser, surtout lorsque les structures observées sont de très petite taille.

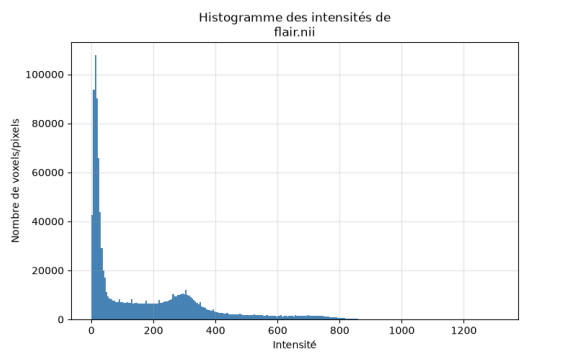
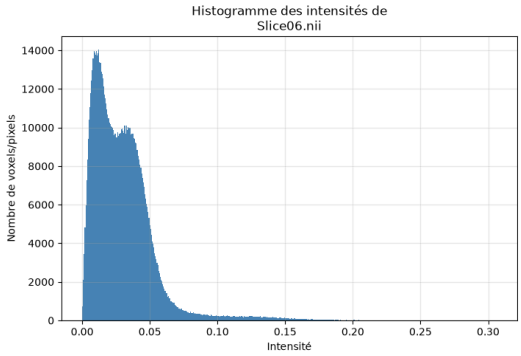
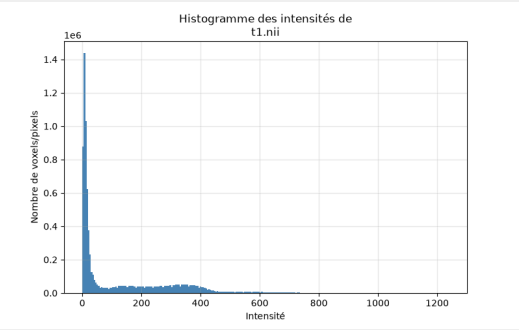
	Effet de volume partiel ? Si oui, où ?
Epaule.pgm	Epaule.pgm ne présente pas d'effet de volume partiel évident.
rat111.vff	Le scan du rat ne présente aucun effet de volume partiel. Les images sont très propres à ce niveau.
Flair.nii	La coupe axiale ne présente pas d'effet de volume partielle contrairement aux coupes sagittales et coronales. Ces deux dernières coupes sont difficiles voir impossibles à utiliser pour une analyse visuelle. 
Heart (Slice06.nii)	Il y a définitivement de l'effet de volume partielle. La coupe axiale est la coupe ayant le moins de cet effet quoi qu'il y en a tout même surtout au niveau du cœur. Les deux autres axes sont entièrement inutiles pour une analyse concrète puisque les voxels sont de trop grosse taille par rapport aux structures.

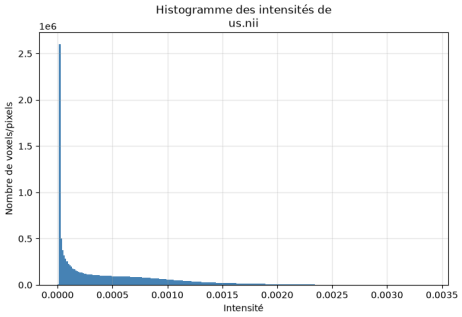
	
T1.nii	T1.nii est l'un des lots de données de la plus haute qualité visuel. Considérant que la structure visée par ce scan est le cerveau, je ne perçoit pas d'effet de volume partiel sur tous les types de coupes.
us.nii	L'entièreté des images semblent montrer un certain niveau de volume partiel. La qualité des ultrasons est si basse qu'il est difficile de distinguer les structures autres que les structures englobantes comme le cerveau, la bouche ou la gorge. 

v) et vi) Nature du bruit et uniformité

En raison de la nature des images d'imagerie médicale, il y a beaucoup de voxels ayant une intensité de 0. Dans mon code, je limite le nombre de valeurs égales à 0 à 5 000 afin de rendre l'histogramme lisible, mais il faut garder en tête qu'il y a beaucoup plus de noir dans les images que ce que les histogrammes présentent.

	v) Nature du bruit dans chaque image ? (Justifier)
Epaule.pgm	Le bruit semble représenter un bruit Ricien assez standard. 
rat111.vff	Le bruit semble Ricien , car les valeurs tendent vers les positifs sans former une butte centrale associée au bruit gaussien. 
Flair.nii	Pour la même raison que rat111.vff, le bruit semble Ricien aussi.

	
Heart (Slice06.nii)	Ici nous avons trois pic descendant de façon graduel à partir du 0. Cela semble représenter du bruit Ricien aussi. 
T1.nii	Ici, les données sont surtout regroupées proche du zéro avec une longue “queue” s'étendant vers les plus hautes intensité. En analysant les images, il est possible de dire que le bruit est probablement Ricien. 
us.nii	L’histogramme de l’ultrason présente des caractéristiques similaires au T1.nii, le bruit est donc probablement Ricien aussi.

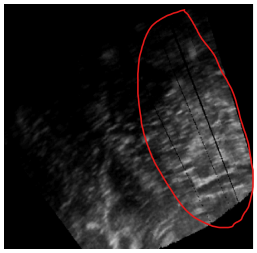
Pour toutes les images, le bruit ne semble pas être uniforme, car il n'est que visible sur les structures imagées et n’est jamais visible au fond noir des images.

vii) Présence d'artéfacts

Voici une définition que j'ai trouvée en ligne au sujet des artéfacts :

« En imagerie médicale, un artéfact est une anomalie visuelle ou une fausse image présente sur un examen (comme un scanner, une IRM ou une radiographie) qui ne correspond pas à une structure anatomique réelle ou à une maladie. » (https://readingscan.com/glossary/artifact)

Sur la base de cette définition, j'ai fait de mon mieux pour tenter d'identifier des artéfacts dans les images.

	vii) Y-a-t-il des artéfacts dans les images ?
Epaule.pgm	Non, aucun artéfact ne semble visible.
rat111.vff	Non, aucun artéfact ne semble visible.
Flair.nii	Non, aucun artéfact ne semble visible.
Heart (Slice06.nii)	Il ne semble pas avoir d'artéfacts, mais il y a définitivement beaucoup de bruits.
T1.nii	Oui, le patient semble avoir bougé pendant l'IRM.
us.nii	Oui, il y beaucoup d'artéfacts visibles sous la forme de traits noirs. La cause potentielle serait possiblement en lien avec les paramètres choisis par l'utilisateur de l'appareil. 

viii) SNR: *Signal to Noise Ratio*

J'ai calculé le SNR de deux façons.

Ma première méthode est la méthode standard, qui consiste à calculer la moyenne de l'ensemble des intensités de l'image, puis à la diviser par l'écart-type du bruit estimé à partir des 10 % des voxels ayant les plus faibles intensités. Cette méthode tient compte de tous les voxels de l'image, incluant le fond noir, qui a tendance à fausser le SNR, surtout dans le cas de nos images où le bruit est seulement présent sur les structures numérisées et n'affecte pas le fond noir.

Par conséquent, ma deuxième méthode reprend le même principe, mais en excluant d'abord tous les voxels dont l'intensité est nulle, soit le fond noir des scans.

En comparant les deux résultats, on est capable d'évaluer l'influence du fond sur l'estimation du rapport signal sur bruit. Généralement, le SNR calculé sans le fond noir devrait être plus représentatif de la qualité des structures anatomiques présentes dans l'image.

Aussi, à des fins informatives, un faible SNR (proche de 0) représente un signal très bruité, tandis qu'un SNR élevé représente un signal clair et fort par rapport au bruit.

	SNR	**SNR ignoring background**
Epaule.pgm	inf	3.416185
rat111.vff	-35.173374	135.293917
Flair.nii	52.723660	69.914544
Heart (Slice06.nii)	17.055798	17.055798
T1.nii	87.512714	116.399757
us.nii	inf	144093002063628544.000000

Trois valeurs se distinguent particulièrement.

Tout d'abord, il y a les valeurs infinies pour Epaule.pgm et us.nii. Une valeur infinie implique que le bruit estimé dans l'image est proche de 0 (voire nul), tout en ayant un signal positif. Cela est particulièrement intéressant dans le cas de Epaule.pgm, puisque cette valeur infinie est seulement présente lorsque le fond de l'image est pris en compte. Cela appuie mes analyses précédentes sur la nature du bruit, à savoir que celui-ci est non uniforme et principalement présent sur les structures numérisées.
Finalement, il y a le SNR négatif de rat111.vff. Cette valeur implique que, dans notre équation du SNR, nous nous sommes retrouvés avec un signal ou un bruit de valeur négative. D'après mes recherches en ligne, cela semble survenir fréquemment dans les scans CT en raison du bruit, du filtrage ou des artéfacts de reconstruction, lesquels sont présents dans le fond noir du scan, considérant que sans le fond noir, le SNR semble normal.

Question 3

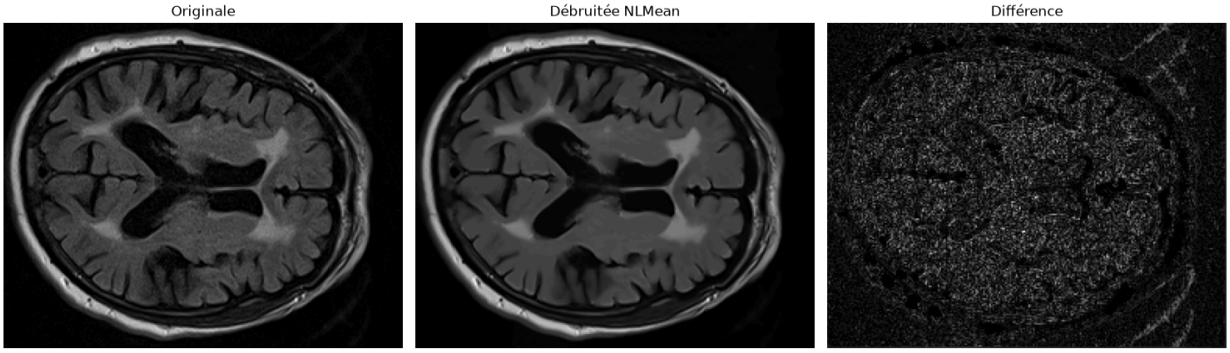
Numéro 3

Méthode 1: NLMeans

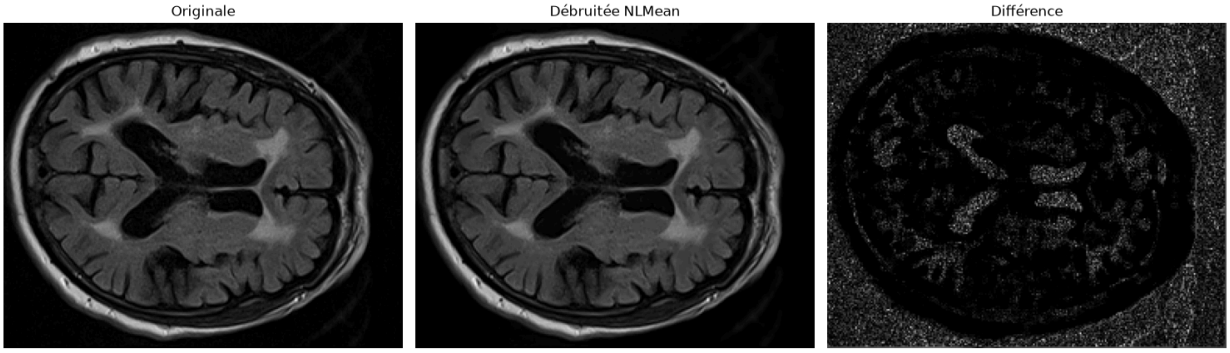
Comme nous l'avons vu en classe, NLMeans est une méthode de débruitage qui se base sur les redondances de motifs présentes dans l'ensemble de l'image plutôt que de se limiter aux pixels voisins. Sachant cela, NLMeans risque de très bien fonctionner sur les IRM du cerveau puisqu'il y a beaucoup de redondances dans les structures cérébrales, particulièrement au niveau de la matière blanche.

Cependant, une valeur élevée de h risque d'être requise, car les IRM sont très bruitées au niveau des structures. Avec une faible valeur de h, seul le fond noir est affecté de façon significative par l'algorithme de débruitage.

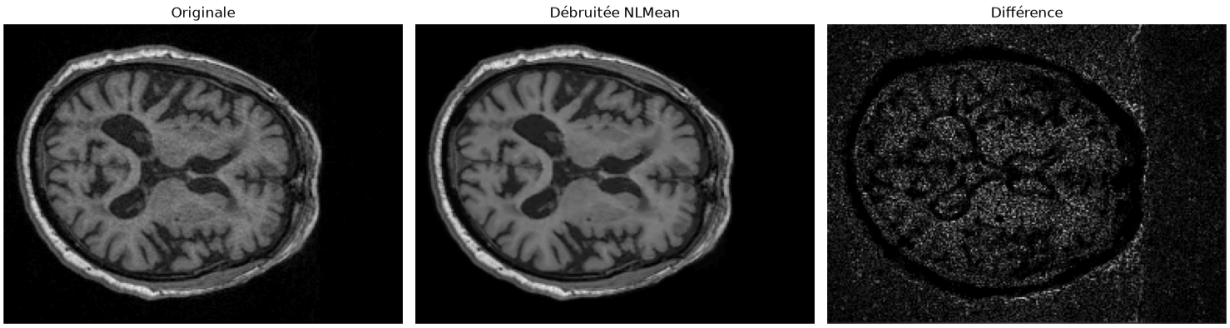
IRM/Flair h=2:



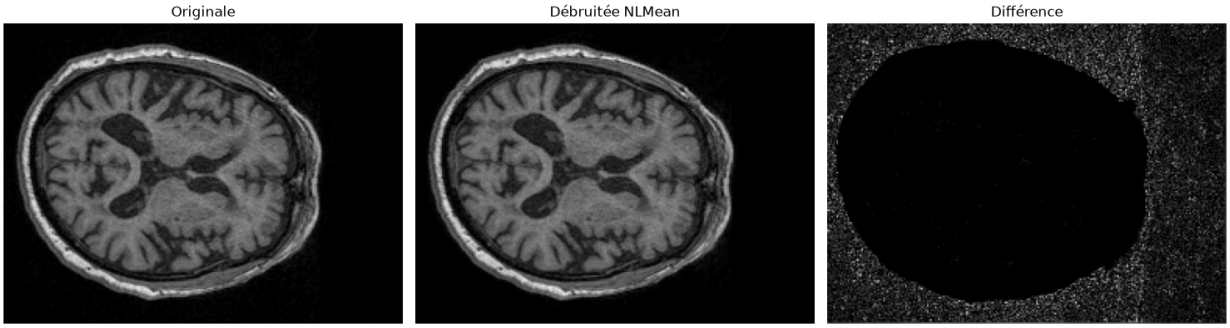
IRM/Flair h=0.8:



IRM/T1 h=2:



IRM/T1 h=0.8:



Les valeurs de paramètres choisies pour débruiter ces images furent les suivantes :

Pour h, soit la force du débruitage, on doit s'assurer de prendre une valeur moyenne afin de conserver les détails tout en réduisant le bruit de l'image. Une valeur plus élevée de h a davantage d'impact sur nos IRM en raison de leur nature très bruitée.
En effet, on voit clairement que seul le bruit du fond noir est affecté lorsqu'on choisit une faible valeur de h pour t1.nii. Cependant, cela semble être insuffisant pour flair.nii, car une grande partie du bruit présent dans les structures demeure visible.

Pour le patch_size, soit la taille des régions considérées par l'algorithme, il faut encore choisir une taille moyenne afin de conserver une performance raisonnable et d'éviter que l'algorithme ignore le bruit ou, à l'inverse, sur-débruite l'image. J'ai donc choisi un patch_size de 5, ce qui équivaut à une fenêtre de 5 × 5 pixels.

Pour le patch_distance, soit la distance maximale de recherche autour d'un pixel, il faut encore faire un équilibre entre la vitesse d'exécution et la qualité du débruitage. J'ai choisi un patch_distance de 15 afin de demeurer raisonnable, mais cette valeur peut facilement être augmentée davantage selon la puissance de la machine utilisée.

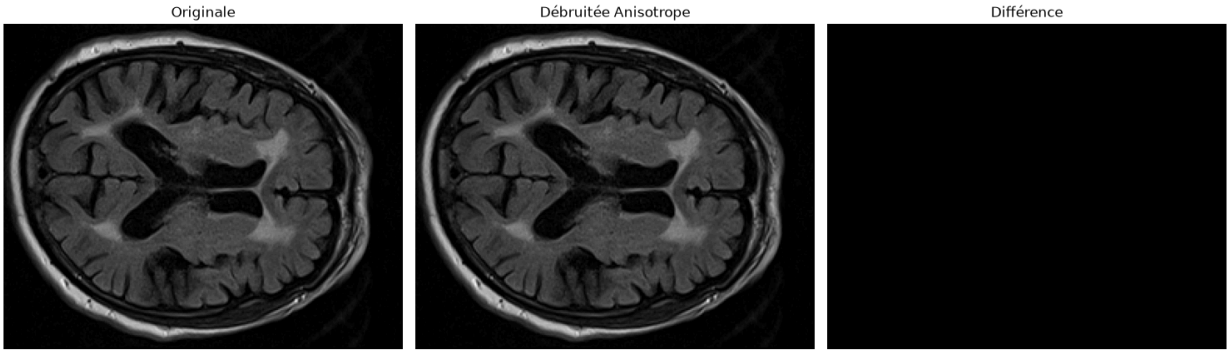
Méthode 2: Diffusion anisotrope

J’ai choisi comme deuxième méthode la diffusion anisotrope afin de confirmer mes observations de la question 2.

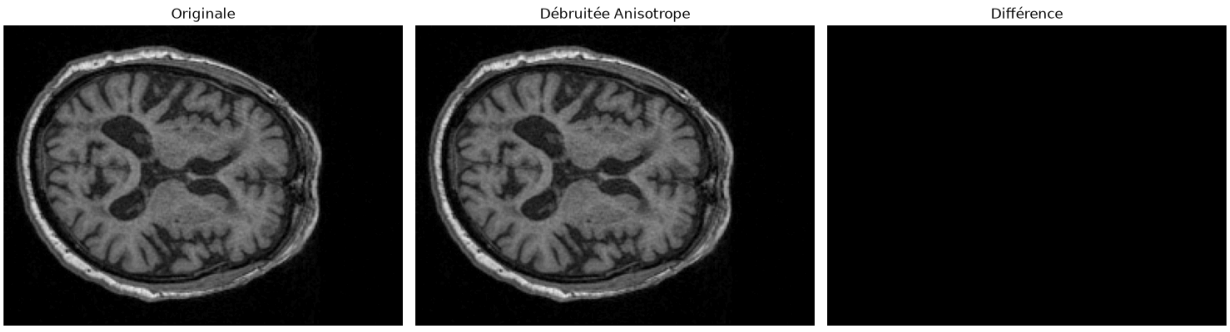
Comme nous l'avons vu en classe, cette méthode de débruitage agit comme un filtre passe-bas et a donc généralement un meilleur effet sur les bruits de type gaussien que sur les bruits riciens. Il est donc possible de ne voir aucune modification significative dans les images si le bruit est trop bien mélangé au signal, ce qui semble être le cas pour flair.nii et t1.nii.

Tel que discuté, on observe en effet que le filtre de diffusion anisotrope n’a aucun effet apparent sur le bruit « caché » de ces images :

IRM/Flair:



IRM/T1:



Pour mes paramètres, nous pouvons modifier le nombre d’itérations effectuées par l’algorithme (nbre_iter) ainsi que la valeur de gamma, soit la vitesse ou l’intensité de la diffusion appliquée par l’algorithme. J’ai choisi des valeurs élevées pour chacun de ces

paramètres (nbre_iter = 500 et gamma = 5), mais le bruit est tellement bien camouflé dans le signal que cela n'a aucun impact visible.

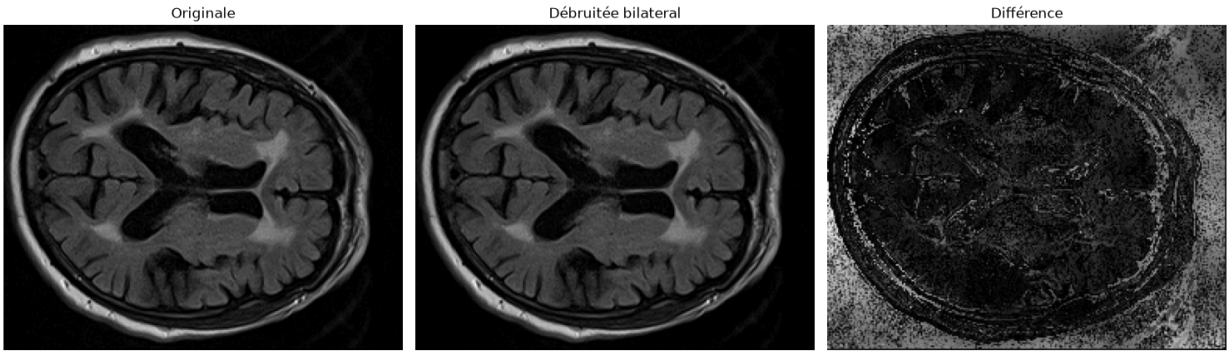
Aussi, puisque cette méthode utilise la diffusivité de Perona-Malik, il n'y a pas de paramètre kappa à considérer.

Méthode 3: Bilateral

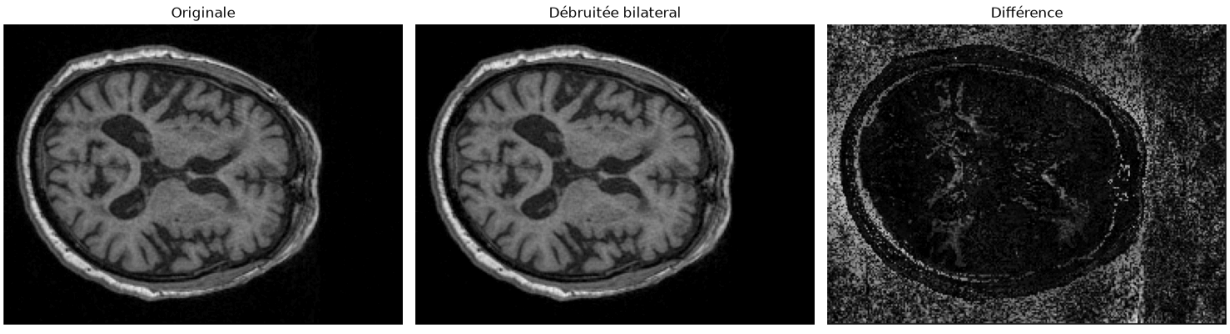
La méthode de débruitage bilatéral cherche à lisser l'image tout en préservant les contours. Comme nous l'avons vu en classe, elle peut être résumée comme un filtre gaussien localisé qui tient compte à la fois des distances spatiales et des différences d'intensité.

Sur des IRM, cette méthode risque de donner des résultats prometteurs pour les structures analysées. On devrait observer une réduction du bruit tout en conservant les structures importantes, telles que les vaisseaux sanguins ou les ventricules.

IRM/Flair:



IRM/T1:



Pour obtenir ces résultats, j'ai utilisé les valeurs de paramètres suivantes :

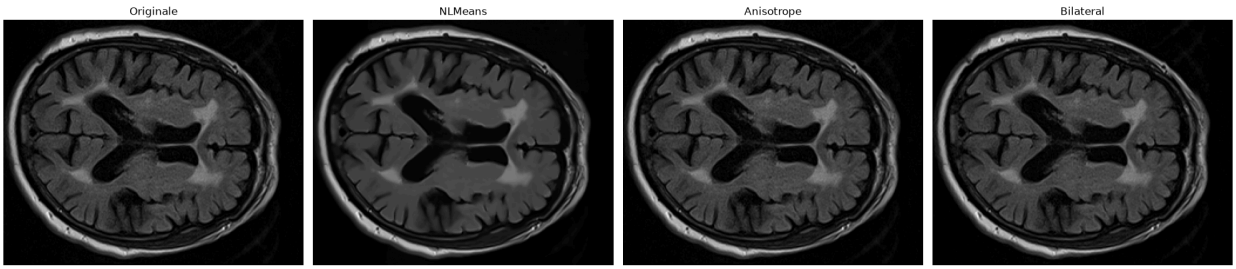
Pour sigma_color, j'ai choisi une valeur de 0,05. Ce paramètre détermine si deux pixels ont des intensités similaires. En d'autres mots, il indique à l'algorithme si un groupe de pixels forme une structure.

La taille de ce groupe de pixels est déterminée par l'autre paramètre, sigma_spatial. J'ai choisi une valeur de 12. Par conséquent, les structures plus petites que 12 pixels risquent d'être atténuées, voire supprimées.
Il est très facile de se tromper dans le choix de ces paramètres.
Cependant, bien que les résultats obtenus avec cette technique soient prometteurs, cette méthode de débruitage présente deux limites.
Premièrement, un bon choix de paramètres requiert une bonne connaissance de l'anatomie pertinente au scan ainsi que de la taille des structures présentes, exprimée en nombre de pixels. Cela est lié à la deuxième limitation, qui réduit l'efficacité de l'algorithme si la résolution du scan est trop faible pour permettre l'utilisation d'une valeur de sigma_spatial raisonnable pour le débruitage (c'est-à-dire lorsqu'il n'y a pas suffisamment de pixels pour représenter adéquatement les structures pertinentes).

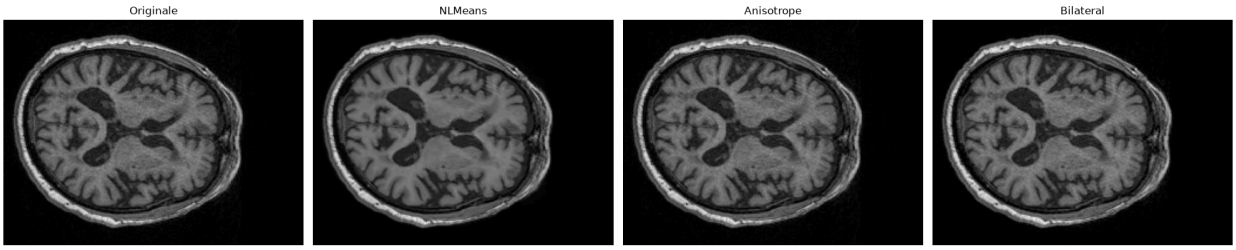
Conclusion sur les méthodes

Voici une comparaison directe entre chaques méthode implémenter

IRM/Flair:



IRM/T1:



En regardant les résultats de mes implémentations, il est difficile de voir une méthode clairement meilleure que les autres.

Cependant, il est certain que la diffusion anisotrope n'est pas un bon candidat à cause de la nature du bruit de ces images. Le bruit est trop bien mélangé avec le signal pour qu'il ait un effet.

Par conséquent, les méthodes désirables serait soient Bilatéral ou NLMeans et j'aurais personnellement tendance à dire que NLMeans est supérieur dans ce contexte à Bilatéral puisque:

- Bilatéral n'est pas capable de prendre en compte les similarités présentes dans le signal de l'image. Elle applique les mêmes transformations partout dans l'image.
- NLMeans est un peu moins affectée par la résolution des images par rapport à la taille des structures.
- NLMeans offre un meilleur contrôle sur ces paramètres.

Notes sur la portion programmation

Creation de view_image.py

J'ai approché le développement du viewer de façon itérative. J'ai implémenté des versions primitives, mais fonctionnelles, du viewer que j'ai ensuite améliorées progressivement (par exemple, afficher une seule coupe à la fois, puis plus tard afficher les trois coupes simultanément).

Dans mon implémentation initiale de base, j'ai remarqué que les dimensions associées aux axes des images étaient différentes. Comme nous l'avons vu en classe, cela est dû au fait que les scans effectués selon certains axes peuvent être acquis à des résolutions plus faibles que les autres, ce qui entraîne des voxels non cubiques.

Initialement, les images étaient présentées selon leurs résolutions respectives. Par exemple, pour le fichier Data_TP0\IRM\Brain\flair.nii, les coupes sagittales et coronales étaient affichées à leur résolution native, soit beaucoup plus faible que celle de la coupe axiale. ~~En spécifiant aspect="auto" dans les appels à imshow, la taille des images est adaptée à la dimension de la fenêtre de l'application.~~ Initialement, j'utilisais aspect="auto", ce qui permettait d'agrandir les images peu importe leur résolution, mais créait également des rapports d'aspect différents entre les axes x et y. Pour éviter ce problème, j'ai adopté une approche différente en utilisant aspect="equal" avec extent=axe_extent. axe_extent est calculé à partir des dimensions physiques de l'image, obtenues grâce à la taille des voxels et au nombre de voxels dans chaque direction. Cette approche permet de conserver une résolution visuellement acceptable tout en respectant les proportions réelles de l'image originale.

~~Aussi, les images sagittales et coronales étaient inversées (le haut de la tête se retrouvait au bas de l'image). Cela est probablement dû à une caractéristique de la machine utilisée pour les scans. J'ai ajouté l'argument --flip si l'utilisateur a besoin d'inverser les images affichées.~~

Après les précisions que tu m'as apportées en classe et par Teams, j'ai exploré plus en détail les en-têtes et les fonctions de Nibabel. J'ai finalement découvert la fonction nib.as_closest_canonical(img), qui réoriente les images selon le standard d'affichage du format NIfTI, soit RAS (Right-Anterior-Superior). Cette exploration m'a été utile pour la question 1c.

Pour la question 1c, j'ai dû ajouter une nouvelle fonction à mon viewer afin de vérifier l'orientation des images selon MrInfo. Cela a été un peu compliqué à faire fonctionner au départ puisque, travaillant principalement sous Windows, j'ai dû installer MSYS2 afin d'installer MRtrix3. Cependant, même après cette installation, MrInfo continuait à me retourner les mêmes erreurs jusqu'à ce que je réalise qu'il n'utilisait pas l'installation de MRtrix3 que je venais d'effectuer.

J'ai ajouté à argparse l'argument --mrinfo afin de faciliter l'exécution du code sur ma tour, sur laquelle je ne voulais pas installer MRtrix3.

Une fois que tout fonctionnait, il ne restait plus qu'à lire la matrice de transformation des images et à interpréter leur orientation à partir des signes présents dans celle-ci [détailler comment interpréter les signes et ajouter la source].

Creation de compute_mIP_MIP.py

Rien de particulier à dire sur l'implémentation. Le principal de mon explication se trouve plus haut dans le rapport, à la question 1b. Cependant, en travaillant sur cette partie, j'ai eu l'idée d'ajouter un paramètre à l'argparse du viewer ainsi qu'au code d'affichage afin de permettre l'affichage des images en niveaux de gris (grayscale).

Creation de show_image_stats.py

Cette partie du travail m'a pris beaucoup de temps à coder, car elle m'a demandé de revisiter mon viewer ainsi que compute_mIP_MIP.

Afin de faciliter la gestion du chargement des données, j'ai créé un fichier utils dans lequel j'ai regroupé toutes les fonctions nécessaires à l'interprétation des types de fichiers et au chargement des données. Dans ce module, j'ai également ajouté l'implémentation nécessaire pour prendre en charge les fichiers PGM.

Aussi, au cours du développement, je me suis rappelé que le format VFF devait être converti vers le format NIfTI afin de pouvoir récupérer les informations nécessaires à la réalisation du travail. ~~En effectuant des recherches en ligne, j'ai réalisé que ces fichiers nécessiteraient une solution personnalisée. Par conséquent, au moment d'écrire ce paragraphe, j'ai décidé d'ignorer cette partie du travail afin de m'assurer d'avoir suffisamment de temps pour m'attaquer à la question 3.~~

En discutant avec des collègues, j'ai appris qu'il était possible d'utiliser Fiji pour convertir les fichiers VFF en NIfTI, ce que j'ai finalement fait.

Creation de denoise_image.py

Avec un view_image.py suffisamment robuste, l'implémentation de denoise_image.py s'est bien déroulée. J'ai demandé à l'IAG de reprendre le code de mon viewer afin de créer une version adaptée à mon argparse.
Après cela, il ne restait plus qu'à trouver et à implémenter les méthodes de débruitage provenant des bibliothèques scikit-image et SciPy, ce que j'avais déjà fait dans d'autres cours de mon baccalauréat.

Utilisation de l'IAG

J'ai utilisé Copilot de quatre façon:

- Pour l'implémentation de base du argparse que j'ai ensuite modifié à mes besoins.
- Pour l'implémentation de partie du code redondante ou pour du formatage de texte dans le terminal.
- À des fins de consultation pour expliquer des concepts sur les méthodes appliquées sur les images dans le contexte de l'imagerie médicale.
- Pour faire les entêtes des fichiers de code terminer.
- Pour corriger l'orthographe de mon rapport après l'avoir écrit au complet.

Je me suis assuré de m'approprier ou d'au moins comprendre le code que Copilot me fournissait.

Fin du rapport